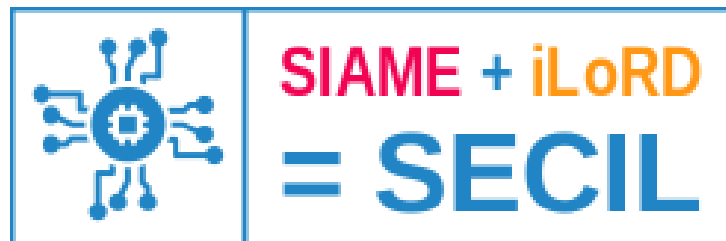


Rapport Projet Synthétiseur

2025/2026



Groupe BotWave :

Arthur Arnold

Noémie Gireaud

Paul Cluzel

Tinhinane Ait-Messaoud

Client :

Emmanuel Rio

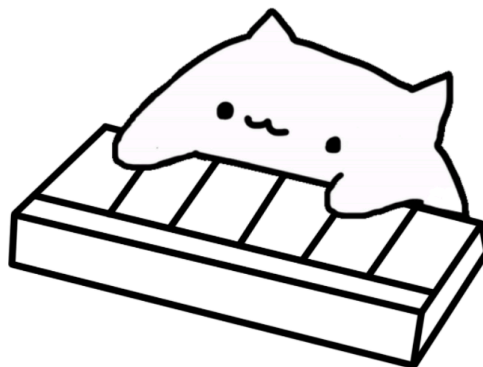


Table des matières

Introduction	3
Description du projet	3
Cas d'utilisation	5
Cas nominaux :	5
Conception de la solution	6
Gestion de projet	7
Étapes de la conception du projet	7
Description du travail réalisé	8
Gestion des entrées	8
Traitement sonore	9
Création d'un oscillateur	9
Manipulation de l'amplitude	10
Addition de plusieurs formes d'ondes	11
Transmission des données avec une STM32F407	12
Transmission des données avec une Raspberry Pi 3	14
État du projet	19
Description et résultats de la campagne de test	20
Déploiement	22
Carte SD :	22
Démarrage:	22
Alimentation :	22
Branchement des câbles :	22
Problèmes rencontrés et solutions.	23
Bilan	25
Bibliographie	26

Introduction

Le projet « Développement d'un Synthétiseur » s'inscrit dans le cadre de l'UE « Projet Collaboratif de Recherche et Développement ». Celui-ci a été proposé par M. Emmanuel RIO, notre client et encadrant, dans un cadre personnel avec pour objectif la **mise en place d'un système de synthèse sonore, basé sur un système embarqué**.

Notre équipe **BOTWAVE** est composée d'Arthur ARNOLD, chef de projet, ainsi que Noémie GIREAUD, Paul CLUZEL et Tinhinane AIT-MESSAOUD.

Ce rapport décrit le déroulé du développement du projet, présente les différents choix effectués, les problèmes rencontrés ainsi que les solutions entreprises afin de pallier ces derniers.

Description du projet

L'objectif principal de ce projet est de **développer un synthétiseur sonore numérique**. Un synthétiseur est un système capable de générer artificiellement un signal audible à partir d'entrées de contrôle. Ce son peut varier en fonction des entrées et peut être paramétrable.

La conception d'un appareil de ce genre nécessite plusieurs étapes :

- La création d'un ou plusieurs oscillateurs (forme d'ondes periodiques).
- Le traitement et la modification de ces oscillateurs (ajouter des effets ou améliorer le rendu)
- La sommation des différents oscillateurs pour obtenir un flux d'échantillons numériques et ainsi une unique forme d'onde représentant le signal sonore de sortie.
- La transmission de cette forme d'onde vers un DAC externe pour convertir les valeurs en signal analogique.
- L'envoi du signal analogique vers un lecteur audio (enceinte, casque, écouteurs, ...)

Le choix des entrées n'ayant pas été imposé, nous pouvions utiliser n'importe quel outil permettant de donner des ordres au synthétiseur. Nous avons finalement décidé de nous baser sur des touches de clavier.

Plusieurs contraintes ont été définies par notre client.

- Une latence de 10 ms ne doit pas être dépassée entre l'appui d'une touche et la génération d'un son audible associé.
- Le traitement du signal doit être géré par une Raspberry Pi. La carte microcontrôleur sera chargée de faire l'interfaçage entre les différentes ressources matérielles et responsable du traitement des informations : récupérer les entrées, générer les oscillateurs et transmettre les valeurs de la forme d'onde produite.
- Il est nécessaire de programmer la Raspberry Pi en baremetal, c'est à dire se soustraire d'un système d'exploitation et ce afin de rebondir sur la question de latence et respecter les contraintes temps réel.

- L'utilisation d'un DAC externe s'impose à nous, car le microcontrôleur à notre disposition ne possède pas de DAC intégré. Le protocole demandé pour transmettre les valeurs de l'onde depuis la Raspberry Pi 3b vers le DAC est le protocole I²S (Inter-IC Sound), conçu spécialement pour la transmission de signaux sonores.

La figure Fig. 1 représente un schéma global de l'outil demandé :

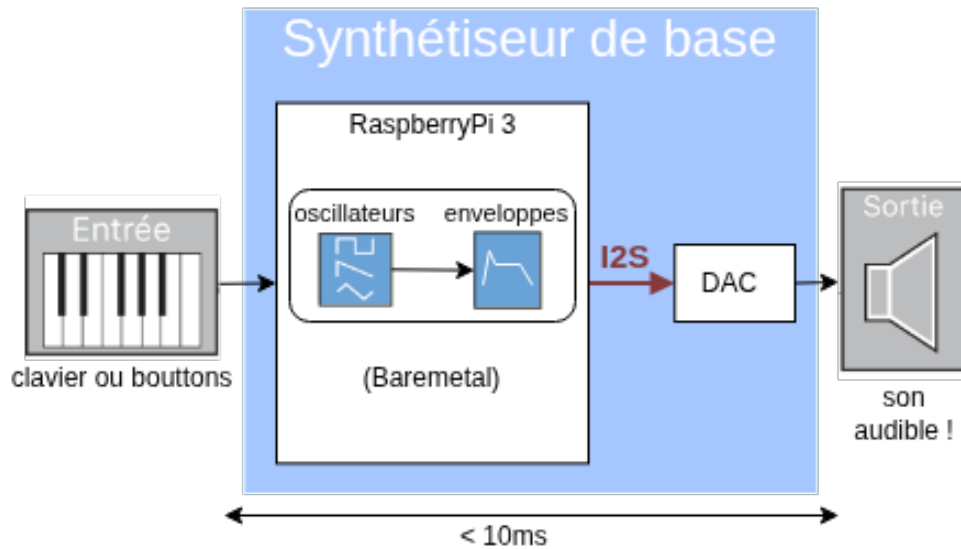


Fig. 1. – Représentation de l'outil demandé

Cas d'utilisation

Notre projet est basé sur différents cas d'utilisations que nous avons documentés durant la période de rédaction du cahier des charges.

Cas nominaux :

- **UC1** : (Synthèse monophonique) L'utilisateur appuie sur une touche associé à une certaine note de musique. Cette note sera audible via une sortie jack durant toute la durée de l'appuie sur la touche.
- **UC2** : (Synthèse polyphonique) L'utilisateur appuie sur plusieurs touches à la fois. Les notes associées son audibles en même temps et leur son se superpose.
- **UC3** : au relâchement de la touche par l'utilisateur, l'arrêt du son est géré par l'implémentation de la phase Release d'une enveloppe d'amplitude de type fondu linéaire, assurant une décroissance progressive du signal jusqu'à ce qu'il devienne inaudible.
- **UC4** : l'utilisateur a la possibilité d'appuyer sur 35 touches présentes jouant ainsi plusieurs notes de plusieurs octaves différentes à la fois.
- **UC5** : Plusieurs styles de son peuvent être entendus lors de l'appui sur les touches. Par exemple, un style ocarina et un style d'orgue.

Conception de la solution

Face aux spécifications et contraintes définies plus tôt, nous avons réalisé plusieurs recherches sur le matériel, modules et protocoles qui seront utiles dans notre projet.

Le matériel utilisé lors de notre projet a été, en partie, récupéré depuis la salle SECIL. Certains éléments ont également été achetés ou fournis par notre client. Le matériel récupéré initialement est le suivant :

- des boutons poussoirs
- un potentiomètre
- un clavier à registre de décalage
- une carte Raspberry Pi Model 3B
- une carte STM32F407
- un DAC PCM5102 (acheté par Mr RIO)
- une enceinte

Nous allons utiliser les modules embarqués suivants :

- le **PCM** (Pulse-Code Modulation) sur Raspberry Pi 3 : un module intégré à la Raspberry Pi chargé de transmettre des données principalement audio. Nous allons l'utiliser pour réaliser la transmission I²S vers le DAC.
- la **DMA** (Direct Memory Access) : nous allons utiliser ce module pour continuellement envoyer nos données contenues dans un tableau vers le PCM. Cela nous permet de libérer le programme principal de cette charge, qu'il pourra, à la place, utiliser pour lire les entrées et générer les données audios.

En ce qui concerne la STM32, le module utilisé pour transmettre des données I²S est le module SPI.

Pour continuer, nous avons défini plusieurs standards que nous souhaitons respecter lors de la création des signaux et la configuration des modules. Cela comprend entre autre :

- L'émission d'un signal stéréo (afin de pouvoir être entendu des deux côtés sur un casque ou des écouteurs),
- L'utilisation d'une fréquence d'échantillonnage de 44100Hz
- L'encodage de chaque échantillon sur 16 bits.

Finalement, en ce qui concerne l'implémentation du programme, celle-ci devra se limiter à de l'implémentation en C sans bibliothèque ou standard importables pour respecter la contrainte baremetal. Une première stratégie d'implémentation que nous avons tout de même prévu est la création de lookup tables. Celles-ci seraient utilisées afin de caractériser le son entendu après l'appuie d'une touche. Elles permettent de se soustraites du calcul des formes d'onde initiales, ce qui diminue la latence et permet de dynamiquement changer d'un son à l'autre. Une autre décision fut également prise pour réaliser l'entièreté des calculs liés au traitement du signal avec des valeurs flottantes entre -1 et 1 et réaliser la conversion entière juste avant la transmission.

Plus d'explications seront apportés sur l'implémentation et l'utilisation des modules dans la section suivante.

Gestion de projet

Pour nous organiser au mieux, nous avons utilisé une **organisation Github** permettant de rassembler plusieurs dépôts pour le *Proof of Concept* (PoC), Le *rendu final* et la *Documentation*. L'organisation possède un **diagramme de gantt** et **tableau kanban** permettant de visualiser et d'attribuer les tâches à faire dans un temps donné.

La communication entre le groupe et le client se faisait depuis un **serveur Discord** permettant de simplifier le dialogue et de montrer l'avancée du projet. De nombreuses réunions ont eu lieu afin de parler des étapes de la conception du projet final.

Le groupe s'est divisé en deux afin de pouvoir continuer de travailler de manière parallèle sur différentes parties tel que le traitement du signal qui s'est réalisé sur STM32 et la partie hardware sur Raspberry Pi.

Étapes de la conception du projet

- Dans un premier temps, tous les membres du groupe ont effectué des recherches de documentations pour comprendre le fonctionnement de la Raspberry Pi mais également de la bibliothèque Circle qui devait être utilisée de base.
- Dans un deuxième temps, nous avons commencé les proofs of concept permettant de tester la faisabilité du projet avec cette bibliothèque qui n'étaient pas concluantes. Alors nous avons décidé de partir sur un kernel basique réalisé par Dr. Poquet que nous avons modifié pour répondre au besoin du projet.
- Les difficultés d'initialisation des modules ont incité le groupe à se scinder en deux parties, afin de garantir une avancée sur le traitement du son sur STM32 et ainsi assurer un rendu final. Une modification complète du kernel a eu lieu pour éviter les blocages des timers, l'initialisation du module PCM. De nombreux tests ont été réalisés durant cette partie pour s'assurer du bon fonctionnement du PCM pour communiquer avec le DAC externe via un oscilloscope. Pour tester un son audible, le signal numérique était calculé d'emblée sous forme d'onde carrée.
- Une fois que le fonctionnement des modules nécessaires de la Rpi garanti, un portage a eu lieu pour intégrer dans le kernel toutes les fonctions sur le traitement du signal afin de gérer des formes d'ondes sinusoïdales, du son polyphonique et de gérer le clavier en entrée.
- Par la suite, nous avons ajouté une forme d'onde pour avoir un timbre différent pour reproduire le son d'un orgue.
- Pour finir, nous avons réalisé une documentation complète pour expliquer le fonctionnement de chaque fonction dans le code présent, ainsi que le poster, la présentation final et le rapport.

Description du travail réalisé

Gestion des entrées

La gestion des entrées n'ayant pas été imposé à l'équipe par le client, nous avons débuté nos expérimentations avec un nombre limité de boutons poussoirs, matériel mis à notre disposition dans notre salle de travail.

Par la suite, l'équipe s'est retournée vers la proposition du client de l'utilisation d'un clavier figurant dans leur inventaire. Ce dernier est un clavier muni de 35 boutons switch, gérés par 5 registres à décalage, montés en série.

L'intérêt de l'utilisation de ces registres est de permettre la centralisation de la lecture de l'état des boutons via une unique sortie SERIAL, commune aux différents registres. Aussi, ceci permet une optimisation des performances en évitant l'initialisation d'un grand nombre de broches, et le transfert d'informations via de nombreux branchements filaires qui, dans l'état courant du projet, ne sont pas stables.

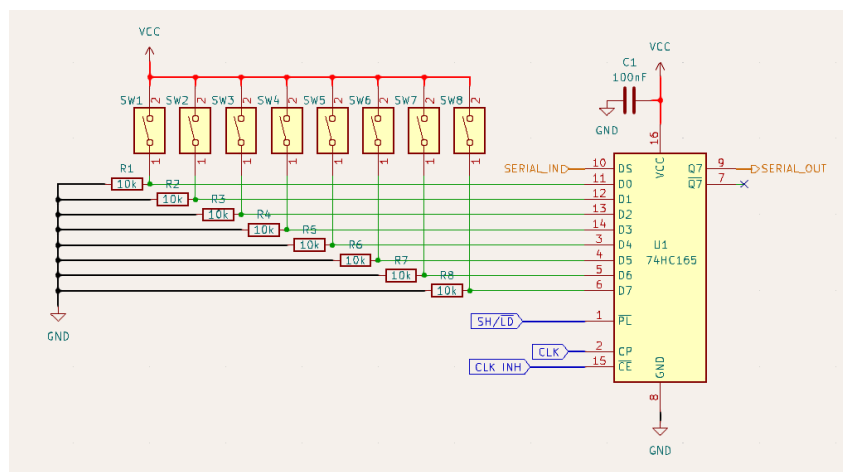


Fig. 2. – Schéma électronique PCB du 1er registre

Le protocole de lecture et fonctionnement du clavier est comme suit :

1. Placer SHIFT/LOAD à 0 afin de charger l'état de chacun des boutons.
2. Replacer SHIFT/LOAD à 1 pour passer en mode décalage.
3. À chaque impulsion d'horloge (broche CLK), que nous faisons varier logiciellement, l'état d'un bouton est renvoyé. Nous bouclons 40 fois afin de récupérer l'état de chacun des boutons dans une variable locale.

Étant donné que nous avons 5 registres à décalage et que chacun gère 8 boutons ($8 \times 5 = 40 \neq 35$), 5 sorties du dernier registres à décalage ne sont liées à rien et devront être ignorées lors de la lecture.

L'ordre d'envoi des boutons est de manière décroissante au niveau de chacun des registres, et croissante entre les registres. Un remaniement de cet ordre dans le logiciel est nécessaire afin de faciliter le traitement de ces entrées et donc le mapping de chaque touche à une note, dans les structures Oscillateur, expliquées précédemment.

Traitement sonore

Création d'un oscillateur

Un son est caractérisé par une forme d'onde. Cette onde peut être périodique ou non, tant qu'elle alterne entre des hauts et des bas. Cette variation doit être comprise entre des bornes spécifiques pour être audible par des oreilles humaines. Pour former un unique son, notre Raspberry Pi doit tout d'abord générer un oscillateur. Un oscillateur représente une forme d'onde périodique. Cette forme peut être sinusoïdale, carrée, triangulaire ou en dent de scie par exemple. Elle caractérise le bruit qui est entendu.

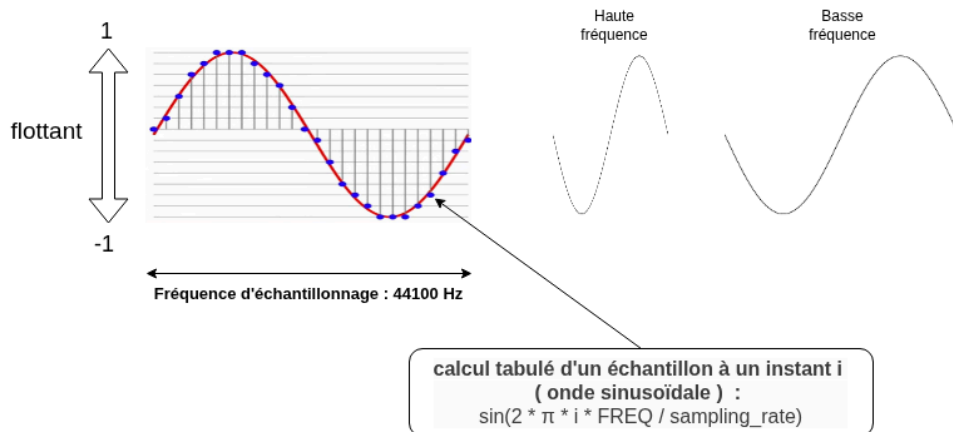


Fig. 3. – Création d'un oscillateur sinusoïdale

La figure Fig. 3 montre un exemple de période d'un oscillateur sinusoïdale. Comme nous pouvons le voir sur cette figure, nous recueillons régulièrement des échantillons de cette courbes ce qui nous permet de la caractériser. Ces échantillons représentent des valeurs flottantes entre 1 et -1. Nous avons décidé d'utiliser une fréquence d'échantillonnage de 44100Hz, ce qui signifie que pour 1 seconde de son, nous enregistrons 44100 échantillons. Pour suivre l'exemple d'un signal sinusoïdal, la formule utilisée est la suivante :

$$\text{sample} = \frac{2\pi * i * \text{FREQ}}{\text{sampling_rate}}$$

avec :

- **sample** : l'échantillon à un point i
- **FREQ** : la fréquence du signal
- **sampling_rate** : la fréquence d'échantillonnage (44100 dans notre cas)

En ce qui concerne la fréquence du signal, celle-ci caractérise la longueur de la période. Un signal avec une fréquence élevée aura une période plus courte, et sera représenté par un son plus aigu tandis qu'une fréquence plus faible générera un son plus grave. Bien entendu, nous avons alloué une fréquence différente à chaque touche de notre clavier.

Dans le contexte de notre projet, nous ne calculons pas manuellement la valeur de chaque échantillon. Cela est lié au fait que nous n'avons pas accès à la fonction sin dans notre Raspberry Pi baremetal. A la place, nous avons décidé de tabuler la suite d'échantillon caractérisant notre oscillateur. Cela apporte plusieurs avantages comme économiser du temps de calcul (surtout pour des flottant qui ajoutent de la latence),

mais également ouvre une porte sur la création de nombreux son différents, qui peuvent chacun avoir leur propre tableau associé.

Une fois l'oscillateur généré, nous pouvons directement convertir les valeurs flottantes en entiers signés de 16 bits et remplir petit à petit un buffer de donné. Ce buffer est récupéré par la DMA qui s'occupe de les transmettre au module chargée de la transmission I²S. Cependant, le son généré par un seul oscillateur seulement est très limité et il manque d'effet pour le rendre plus naturel à l'oreille.

Manipulation de l'amplitude

Une enveloppe est un premier effet de base pouvant être ajoutée à un oscillateur. Celle-ci manipule l'amplitude de la forme d'onde.

L'amplitude représente la hauteur atteinte par la forme d'onde. Plus celle-ci se rapproche des bornes définies (dans notre cas, -1 et 1) alors plus l'amplitude est élevée, et par conséquent le son entendu est fort. A l'opposé, une forme d'onde dont les bornes supérieur et inférieur se rapprochent plus de 0 va avoir une amplitude faible et donc un son moins audible. La figure Fig. 4 représente les deux états décrits précédemment.

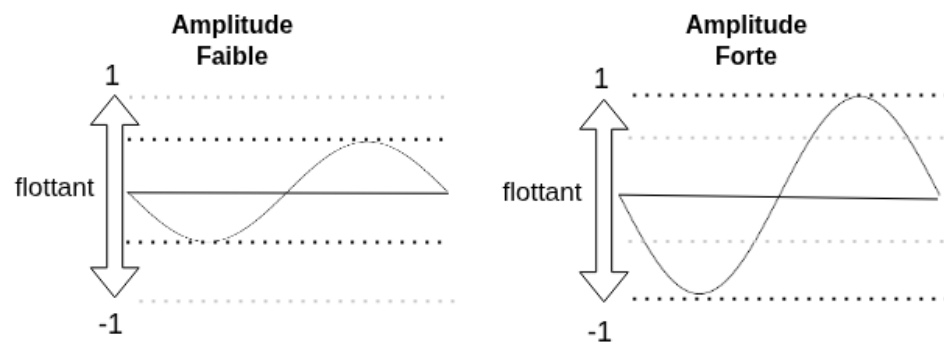


Fig. 4. – Différence entre amplitude forte et faible

La manipulation de l'amplitude peut être utilisée pour ajouter des effets de fondu en entrée et en sortie de son. Ces effets ont pour but de faire progressivement monter le volume du son lorsqu'il arrive, et de le diminuer petit à petit lorsqu'il s'arrête. Il existe de nombreuses stratégies de fondues dont la plus connu : l'enveloppe ADSR (voir Fig. 5)

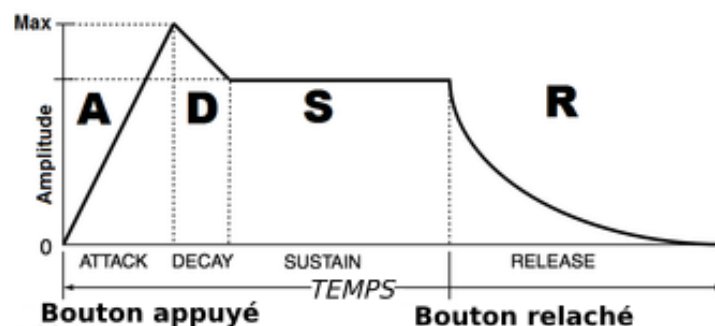


Fig. 5. – Enveloppe ADSR

Cependant, c'est l'enveloppe linéaire que nous avons implémenté dans notre projet. Comme nous pouvons le voir dans la figure Fig. 6 celle-ci se caractérise par une courbe droite pour l'augmentation et la diminution de l'amplitude.

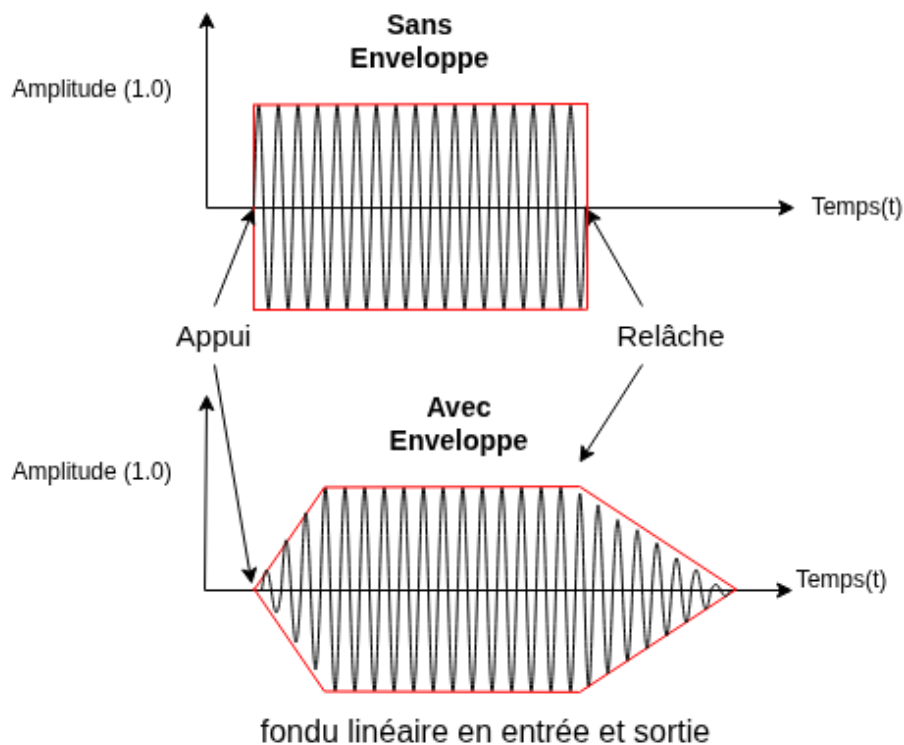


Fig. 6. – Enveloppe linéaire

Addition de plusieurs formes d'ondes

Après avoir ajouté des enveloppes sur nos oscillateurs, nous avons implémenté la fonctionnalité pour en entendre plusieurs à la fois. Cela complète le fait qu'une personne appuie sur plusieurs touches de clavier en même temps. Comme il n'est pas possible d'envoyer plusieurs signaux sonores vers notre DAC, nous devons additionner les oscillateurs afin que l'effet des son superposé soit audible. En réalité, seul une seule forme d'onde en ressort mais, grâce aux miracles de la nature, notre cerveau est capable de la décomposer et d'extraire par lui mêmes les formes d'ondes originales. La figure Fig. 7 montre deux exemples d'addition de plusieurs oscillateurs.

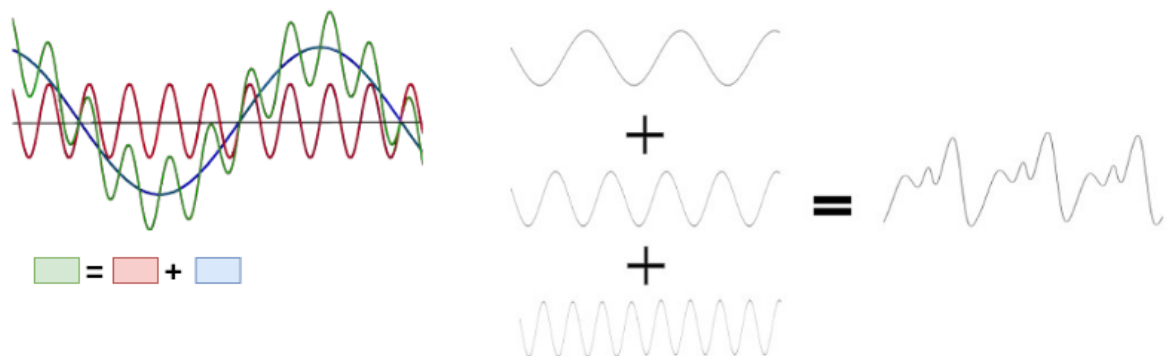


Fig. 7. – Exemples d'addition d'oscillateurs

L'addition de formes d'ondes génère, néanmoins un problème. En effet, si on additionne deux ondes bornées entre -1 et 1 , cela nous offre une nouvelle onde cette fois comprise entre -2 et 2 , ce qui dépasse les bornes définies et ainsi risque de générer un son beaucoup trop amplifié. De plus, diviser la somme obtenue par un nombre d'oscillateur maximum ne suffit pas à traiter les cas de dépassement extrêmes.

Une solution initiale à ce problème fut d'implémenter du « hard clipping ». Si le son additionné dépasse les bornes définies, alors on le limite aux bornes :

$$x = 1 \text{ si } x \geq 1$$

$$x = -1 \text{ si } x \leq -1$$

$$x = x \text{ sinon}$$

Cependant, cette implémentation crée des extrémités très dures, impactant le rendu sonore. Pour contrecarrer cela, nous utilisons à la place une technique de « soft clipping » qui arrondit l'amplitude sur les bornes et conserve une courbe propre en haut et en bas de l'onde :

$$x = \frac{2}{3} \text{ si } x \geq 1$$

$$x = -\frac{2}{3} \text{ si } x \leq -1$$

$$x = x - \frac{x^3}{3} \text{ sinon}$$

Le processus final réalisé dans notre projet en ce qui concerne la création de notre son suit alors les étapes suivantes :

- création d'un oscillateur et ajout de son enveloppe pour chaque touche appuyée ou relâchée
- addition des oscillateurs générés entre eux
- mise en place de l'amplitude globale et du soft clipping pour un rendu plus propre.

Transmission des données avec une STM32F407

C'est grâce à une première expérimentation sur STM32 que nous avons compris les fondamentaux requis permettant d'assurer la transmission de données audio. Cela comprends l'utilisation de la DMA, ainsi que l'initialisation du module chargé de transmettre les données I²S.

1. DMA et Buffer circulaire :

Après l'initialisation de la DMA sur STM32F407, celle-ci reçoit un pointeur vers le tableau contenant les données à transmettre, ainsi que la taille de ce tableau. Ces données sont dynamiquement calculées par le programme principale en fonction des entrées reçues. Afin de continuellement réagir aux entrées et de transmettre en permanence les données audios, nous avons également utilisé une stratégie de buffer circulaire.

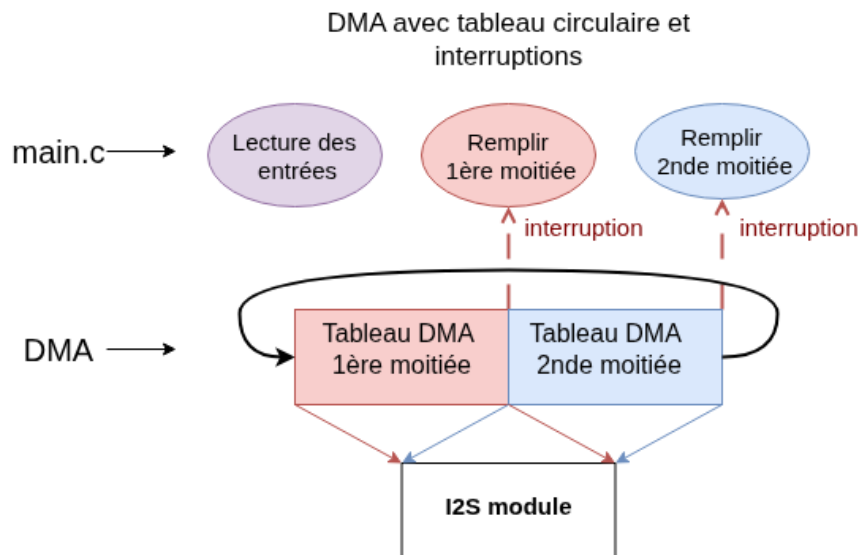


Fig. 8. – Fonctionnement d'un DMA à buffer circulaire sur STM32

Le schéma en Fig. 8 illustre le fonctionnement de notre DMA à buffer circulaire. La DMA est chargée de transmettre les données du tableau vers le module I²S. C'est grâce à son initialisation en mode buffer circulaire qu'elle peut tourner en boucle sur ce tableau et retransmettre les données depuis le début une fois qu'elle a terminée. Comme la DMA ne contient qu'un pointer vers le tableau, le programme peut également y accéder et modifier dynamiquement son contenu pendant qu'elle transmet. Cependant, le programme principale ne devrait pas modifier la section du tableau qui est en train d'être envoyé. Cela risque de créer des désaccords dans la sortie audio et générer un son incorrect. Pour mitiger ce problème, la DMA communique avec le programme principal sous la forme d'interruptions. En effet, elle a également été configurée pour envoyer 2 types d'interruptions, une première après avoir transmit la moitié du tableau, et une deuxième en atteignant la fin, juste avant de reboucler. Ces interruptions sont vitales pour indiquer au programme principale quelle partie du buffer la DMA est entrain de transmettre. Ainsi, après avoir reçu la première interruption, le programme est libre de modifier la 1ère partie du tableau, et après avoir reçu le second type, c'est la seconde moitié du tableau qui peut être modifiée.

Afin de garantir une latence faible, il est préférable de donner à la DMA un buffer le plus petit possible. Dans notre programme, la taille du tableau est de 256 éléments, ce qui signifie 128 éléments sont écrits dans le tableau à chaque interruption de la DMA. Cela permet de plus vite réagir aux entrées. Une dernière chose à prendre en compte est que, à l'opposé du buffer DMA utilisé dans la Raspberry Pi 3b, notre buffer initialisé dans la STM32 inclus des éléments de taille 16 bits et non 32. Cela signifie que durant la phase traitement du signal, chaque écriture du tableau devait être effectuée en double, une première à un indice i pour le channel de droite, et une seconde à l'indice $i+1$ pour le channel de gauche.

2. Transmission I²S

Un dernier aspect à prendre en compte pour assurer la bonne transmission des données est la configuration du module de transmission I²S. Dans le cadre de la STM32F407,

c'est le module SPI qui en est chargé et qui peut être initialisé pour envoyer des données suivant le protocole I²S.

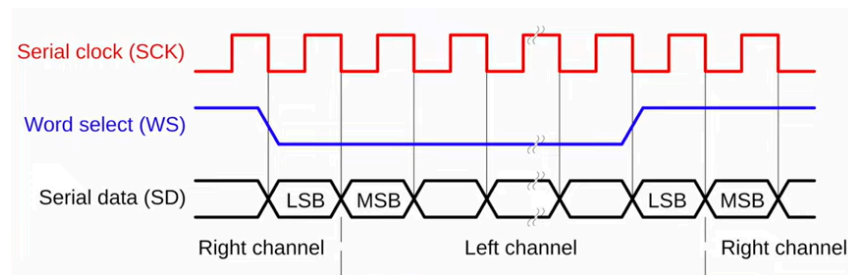


Fig. 9. – Protocole I²S

Comme le montre la figure Fig. 9, le protocole I²S nécessite 3 signaux :

Serial Clock : une clock assurant une communication synchrone entre le module transmetteur et le module receptr. La vitesse de cette clock se calcule en fonction de la fréquence d'échantillonnage, en suivant la formule suivante :

$$\text{SCK speed} = \text{sampling_rate} * \text{sample_bits} * \text{stereo}$$

avec :

- **sampling_rate** : la fréquence d'échantillonnage (44100Hz dans notre cas)
- **sample_bits** : le nombre de bits utilisé pour représenter un échantillon (16 dans notre cas)
- **stereo** : si le son est en mode stéréo (2 channel audio) alors il faut multiplier le tout par 2, sinon 1. Dans le cadre de notre projet, nous transmettons un signal stéréo.

World Select : un signal indiquant la channel en cours de transmission.

Serial Data : la donnée audio à transmettre pour une channel spécifique. Cette donnée est donc caractérisée sur 16 bits représentant un nombre entier signé. Pour rappel, la conversion entre les flottants utilisés pour le traitement du signal et leur valeur entière se fait juste avant l'écriture dans le buffer du DMA.

En ce qui concerne la Raspberry Pi 3b, le module PCM chargé de réaliser la transmission I²S s'initialise différemment que le module SPI de la STM32, mais la configuration de l'échange I²S reste la même.

Transmission des données avec une Raspberry Pi 3

A cause de la différence de hardware entre la STM32F407 et la RaspberryPi 3, le procédé d'initialisation de cette dernière et l'utilisation des modules de transfert tels que la DMA utilise une toute nouvelle logique.

1. Kernel :

Nous avons récupéré le code du kernel réalisé par Dr. Poquet pour initialiser le CPU afin de désactiver la MMU, le cache, les niveaux de privilèges, gérer les interruptions CPU

et de gérer un seul coeur. Il a fallu activer la FPU du processeur (cortex A-53) pour travailler avec des nombres flottants et ajouter la gestion des interruptions provoquées par la DMA.

2. Initialisation et gestion du PCM :

Pour communiquer avec le DAC externe, il faut initialiser le module PCM en plusieurs étapes.

- Choisir l'horloge PLLD pour synchroniser le module
- Initialiser les broches liées au module PCM
- Activer le module PCM
- Paramétrer les registres systèmes du module pour préciser comment traiter les données sur les canaux du I²S et préciser que le PCM doit être le **master** pour gérer le DAC externe.
- Choisir que le PCM doit envoyer des signaux à la DMA (*signal DREQ*) pour informer qu'elle a besoin de remplir sa file de transmission.
- Activer la transmission

8.4.3 DMA

- a) Set the EN bit to enable the PCM block. Set all operational values to define the frame and channel settings. Assert RXCLR and/or TXCLR wait for 2 PCM clocks to ensure the FIFOs are reset. The SYNC bit can be used to determine when 2 clocks have passed.
- b) Set DMAEN to enable DMA DREQ generation and set RXREQ/TXREQ to determine the FIFO thresholds for the DREQs. If required, set TXPANIC and RXPANIC to determine the level at which the DMA should increase its AXI priority,
- c) In the DMA controllers set the correct DREQ channels, one for RX and one for TX. Start the DMA which should fill the TX FIFO.
- d) Set TXON and/or RXON to begin operation.

Fig. 10. – étapes pour activer le PCM

Il est important de bien respecter l'ordre présent sur la Fig. 10 et de respecter les timings avec les cycles des horloges donnés sinon le module ne fonctionnera pas.

Voici la liste des registres à modifier en modifiant les bits permettant d'activer ou désactiver des informations utiles pour le bon fonctionnement du PCM:

noms	information
CS_A	PCM Control and Status
FIFO_A	PCM FIFO Data
MODE_A	PCM mode
TXC_A	PCM Transmit Configuration

DREQ_A	PCM DMA Request Level
CM_PCMCTL	Clock for PCM
CM_PCMDIV	Division of the clock for precision

3. Initialisation et gestion de la DMA :

L'utilisation de la DMA sur Raspberry Pi permet de soulager la charge de travail du CPU en envoyant automatiquement les données du buffer stockées en RAM pour les envoyer au module PCM lorsque la FIFO TX de celui-ci est à moitié vide.

La DMA possède plusieurs canaux mais un seul canal suffit, pour la transmission du PCM. Un canal est dans une zone mémoire qui stocke plusieurs registres hardware qui déterminent le fonctionnement de la DMA. Elle a besoin d'une structure alignée sur 256 bits permettant de gérer d'autres registres hardware qui gère l'adresse source, et le parcourt du tableau avec l'incréméntation, l'adresse de destination et l'adresse d'une structure de contrôle de la DMA lorsque la DMA a fini de parcourir et d'envoyer le nombre d'octet précisé.

Pour reproduire au mieux et faciliter le portage de code de la STM32 vers la Raspberry Pi, nous avons mis en place deux structures de contrôle permettant de traiter la première moitié et l'autre moitié du buffer qui stocke les données numériques. Ces deux structures se pointent entre elles permettant de continuellement d'envoyer les données du buffer de manière circulaire vers le PCM et à chaque changement une interruption est envoyée pour changer la machine à état du programme pour écrire dans la partie du buffer disponible et qui n'est pas utilisé par la DMA.

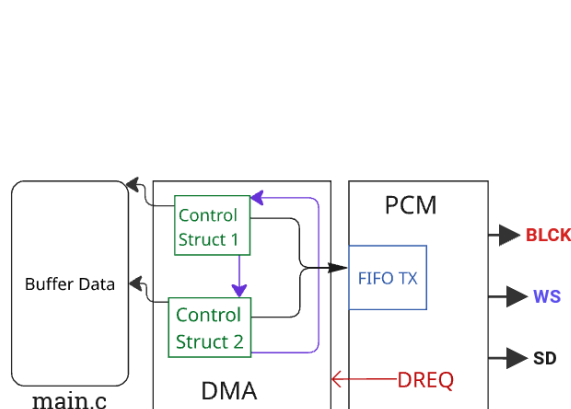


Fig. 11. – Fonctionnement général

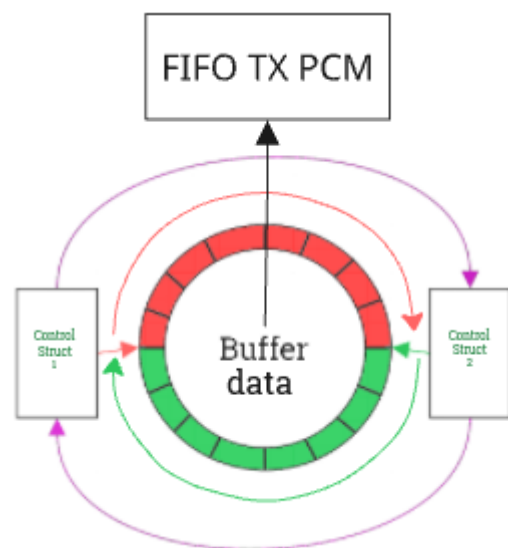


Fig. 12. – Fonctionnement circulaire

Grâce au signal **DREQ** envoyé par le PCM lorsque le nombre d'éléments présent dans sa FIFO est inférieur à la valeur précisé lors de l'initialisation du PCM, la DMA enverra les données jusqu'à ce qu'elle ne reçoit plus le signal.

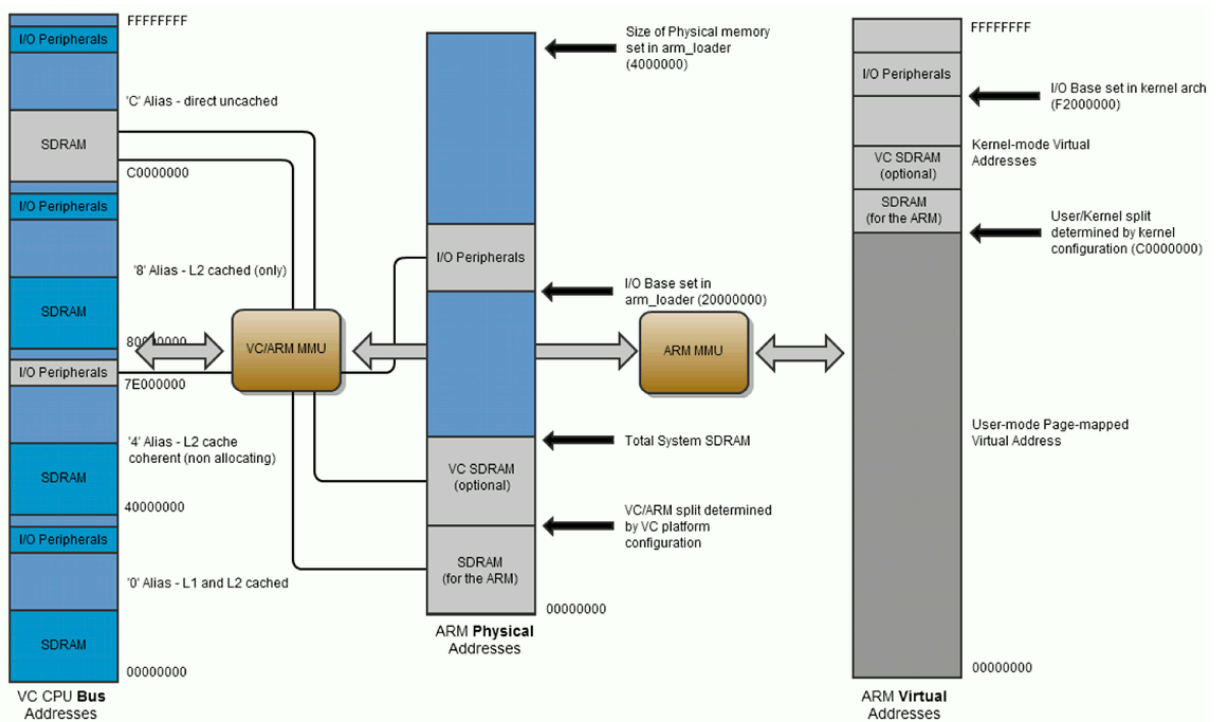


Fig. 13. – mémoire du Raspberry Pi

Nous pouvons voir que la Raspberry Pi possède plusieurs plages mémoire, une représentant le bus système et l'autre accessible par le CPU. Il faut utiliser les adresses physiques pour initialiser la DMA depuis la section I/O peripherals. Afin de lui préciser les adresses des structures de contrôle, des adresses sources et destinations, il faut utiliser la section SDRAM et I/O peripherals de la plage mémoire du bus système car la DMA devient autonome est utilise ce bus plutôt que l'autre réservé au CPU.

De plus, la taille du buffer est la plus courte possible (256 éléments) afin d'avoir la latence la plus courte possible de l'entrée à la sortie audio.

4. Gestion des interruptions :

Les interruptions provoquées par la DMA permettent de savoir sur quelle partie du buffer de données écrire pour stocker les signaux numériques sur la partie non traitée par la DMA.

Les interruptions sont provoquées en niveau de privilège **EL1** et sont gérées par le CPU grâce à la table de vecteurs permettant d'aller dans un handler pour les traiter. Afin de changer la machine à état du programme pour sélectionner la bonne partie du buffer à modifier.

Ainsi avec toutes les étapes réalisées voici la Fig. 14 qui représente le fonctionnement du projet final.

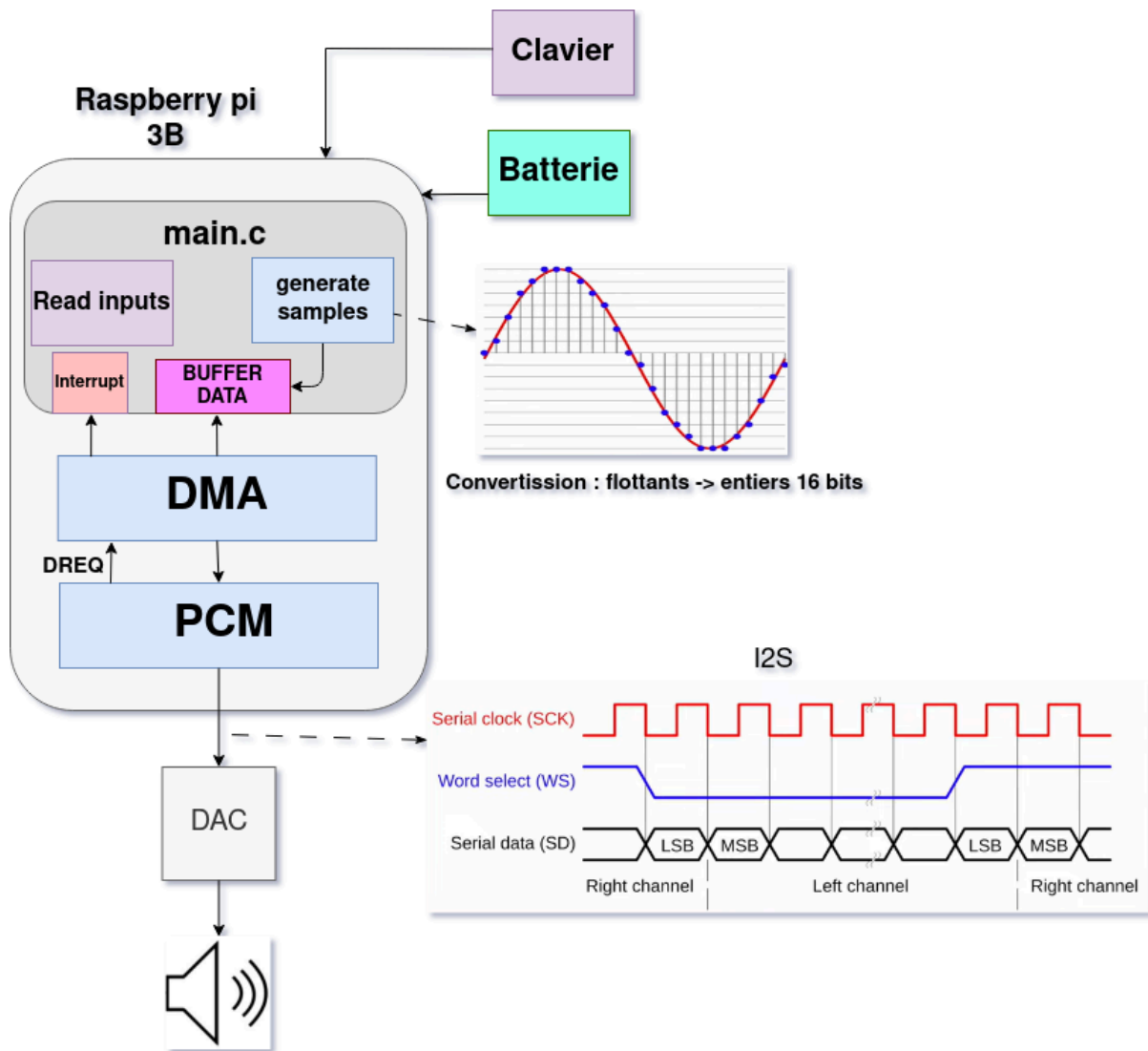


Fig. 14. – Schéma du projet final

État du projet

Nous avons finalisé avec succès ce qui nous avait été demandé par notre client, M. RIO, que ce soit par l'apprentissage des notions de traitement du signal, mais aussi par la conception et la mise en œuvre du synthétiseur sur Raspberry Pi. Le système est capable de générer un signal audio en temps réel à partir d'une entrée utilisateur, en respectant les contraintes de latence et de transmission via le protocole I²S vers un DAC externe. Néanmoins, plusieurs axes d'amélioration restent possibles afin de rendre le produit plus complet et plus proche d'un véritable synthétiseur musical :

- l'ajout d'une fonctionnalité permettant à l'utilisateur de choisir la forme d'onde générée par le synthétiseur (sinusoïdale, carrée, triangulaire, dent de scie).
- la création de soudures propres ainsi que d'un boîtier conçu en impression 3D pour contenir la carte embarquée, les périphériques externes et leurs connexions, permettant une meilleure robustesse, une meilleure protection électronique et une présentation professionnelle du produit.
- l'utilisation d'une enveloppe ADSR (Attack, Decay, Sustain, Release) afin de contrôler l'évolution de l'amplitude du signal dans le temps, ce qui est essentiel pour reproduire fidèlement le comportement réel des instruments de musique et éviter un son artificiel ou brusque.
- la gestion du volume via un composant matériel externe à la Raspberry Pi.
- l'amélioration de la génération sonore par l'ajout d'harmoniques en plus de la fréquence fondamentale, afin de simuler différents instruments.
- l'intégration de nouvelles touches physiques, notamment des touches similaires à celles d'un piano, afin d'améliorer l'ergonomie et de permettre une utilisation plus naturelle et intuitive par l'utilisateur.

Description et résultats de la campagne de test

Afin de vérifier le fonctionnement correct de notre projet, le travail s'est fait de manière incrémentale, avec des phases de tests à chacune de ces étapes. Nous expliquerons dans cette partie les grands axes de développement et tests de notre projet final sur Raspberry Pi.

- La vérification de la bonne configuration des différentes horloges internes nécessaires au fonctionnement du DAC externe avec utilisation d'un Oscilloscope. Nous pouvons voir, dans les figures suivantes, les résultats concluants pour différents signaux d'horloge.

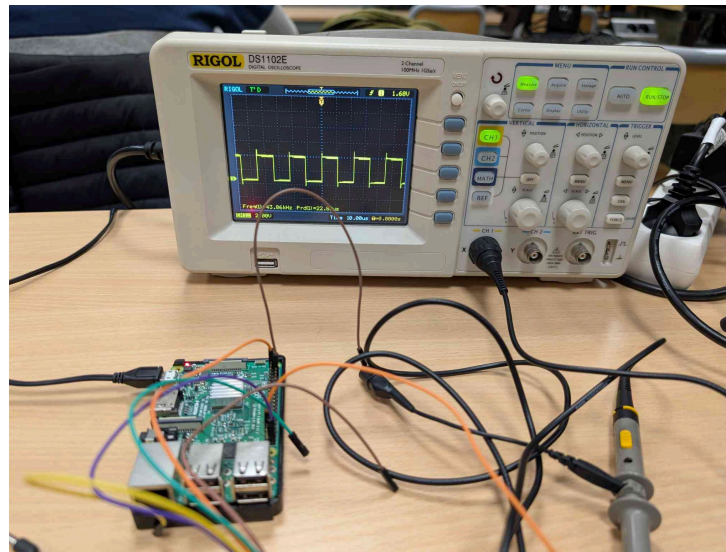


Fig. 15. – Signal d'horloge LCLK

La figure ci-contre met en avant l'horloge à une fréquence proche de 44.1 KHz, qui est la fréquence souhaitable lors de la manipulation de l'audio.

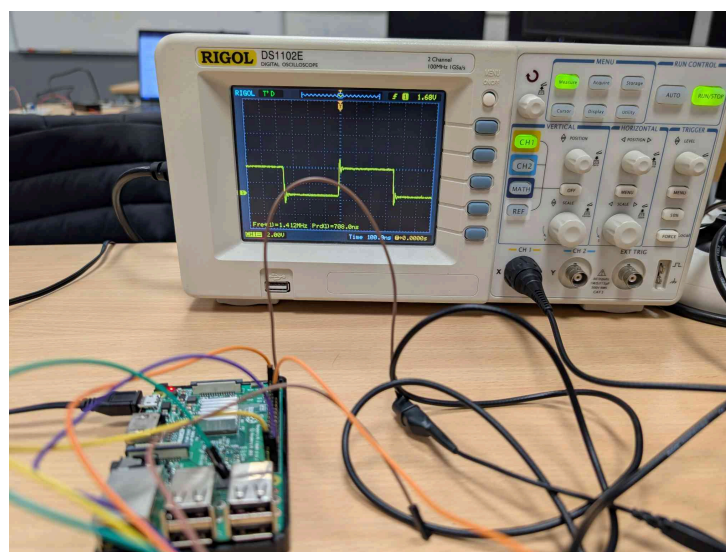


Fig. 16. – Signal d'horloge pour la fréquence d'échantillonnage

Nous pouvons constater une fréquence cohérente au calcul de la fréquence d'échantillonnage.

- La génération d'un signal carré. Dans ce cas, nous vérifions la périodicité et la continuité de ce signal, afin d'attester du fonctionnement du module PCM par le remplissage de sa file manuellement.

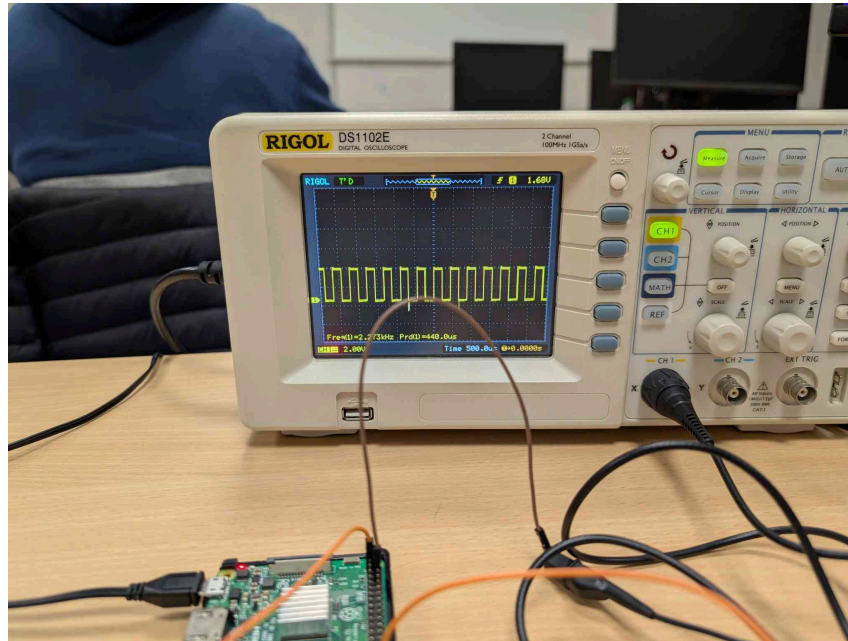


Fig. 17. – Signal d'horloge pour la fréquence d'échantillonnage

L'ajout de l'utilisation de la DMA pour le remplissage du buffer avec des interruptions vient suivre la même analyse que l'étape précédente.

- Analyse de la cohérence de la sortie sonore par rapport au signal de sortie. La vérification de cette étape du projet s'est faite via Audacity afin de vérifier une réelle continuité du signal avec analyse de la courbe sinusoïdale de sortie sonore. Cette analyse a permis de faire ressortir des anomalies dans notre traitement du signal.
- La vérification de l'interaction des entrées clavier avec sorties s'est faite en premier lieu avec utilisation d'une LED externe, puis via l'analyse auditive des sorties sonores sur une partie des touches et enfin l'ensemble des touches.

Déploiement

Carte SD :

Le projet possède un Makefile permettant de compiler le code pour générer une image du kernel.

Le dossier Boot sur le Github possède l'entièreté des fichiers permettant de faire fonctionner la Raspberry Pi 3B au démarrage afin que la machine puisse démarrer avec l'image qui est générée à chaque fois par le Makefile.

Il faut copier ainsi l'intégralité du dossier boot ainsi que l'image **kernel8.img** sur la carte SD qui sera utilisée. La carte SD doit posséder une partition boot au format FAT32.

Démarrage:

Pour s'assurer du bon déploiement du kernel, il faut regarder la LED verte ACT qui doit clignoter une fois pour nous informer que le système a bien trouvé l'image et que la Raspberry Pi 3B boot bien.

Alimentation :

Il faut également alimenter la Raspberry Pi 3B avec une batterie externe ou alors avec un bloc d'alimentation permettant de gérer la masse du secteur pour éviter d'avoir du bruit de 50Hz qui s'ajoute au signaux sonore de l'enceinte. Le bloc ou la batterie doit pouvoir délivrer une tension de 5.1V.

Branchement des câbles :

Pour que le **DAC externe 5102** fonctionne correctement via le Raspberry Pi, il faut relier les broches suivantes entre elles :

Raspberry Pi 3B	DAC externe 5102
GPIO 18 (PCM_CLK)	BCK
GPIO 19 (PCM_FS)	LRCK
GPIO 21 (PCM_DOUT)	DIN
5v et/ou 3v	VIN
GND	GND

Une documentation est présente dans le dépôt github **Documentation** de l'organisation BotWave pour expliquer toutes les étapes utiles ainsi que le code du projet. Les fichiers README aident également lors du montage.

Problèmes rencontrés et solutions.

Nous avons rencontrés de nombreux problèmes durant ce projet mais nous avons pu les résoudre et continuer d'avancer pour finalement rendre un projet complet.

1. Bibliothèque Circle :

Initialement le projet devait utiliser la bibliothèque **Circle** permettant de simplifier le code bare metal et d'initialiser les modules nécessaires de la Raspberry Pi. Malheureusement le code produit était incapable de booter et le kernel ne se lançait pas. Donc pour palier à ce problème nous avons récupéré le code du kernel produit par Dr. Poquet que nous avons modifié afin de pouvoir démarrer sur la machine.

2. Documentation incomplète pour PCM :

Puisque que nous faisons du bare metal, nous avons rencontré un autre problème sur l'initialisation du module PCM et de la gestion des horloges de la Raspberry Pi. La documentation étant incomplète et peu explicite il fut difficile de réaliser cette partie. C'est pourquoi le groupe fut divisé pour continuer d'avancer sur un rendu via une STM32 et de pouvoir commencer à travailler sur le traitement du signal.

En trouvant des bouts de documentations sur d'autres sites et en comprenant mieux les spécifications de la Raspberry Pi, nous avons pu initialiser le module PCM et horloges nécessaires. Grâce à cela, nous avons réussi à le faire fonctionner permettant d'envoyer les signaux numériques via le protocole I²S.

3. Alimentation externe :

Une fois que nous avons réussi à avoir du son, il y avait un bruit parasite d'une fréquence de 50Hz était présent. Ce bruit disparaissait lorsque quelqu'un touchait la Raspberry Pi, le DAC, ou encore l'enceinte. De ce fait, nous avons diagnostiqué un problème de masse. Ce problème se produit à cause du bloc d'alimentation de la Raspberry Pi ne permettant pas de gérer la masse secteur qui venait jusque sur la machine. Nous avons trouvé deux solutions permettant de gérer ce problème. La première solution que nous utilisons et d'alimenter l'ordinateur avec une batterie externe. La deuxième solution possible est d'utiliser un bloc d'alimentation AC/DC gérant la masse secteur et de gérer la tension de sortie à 5.1V. (*Mean Well RS-25-5 LED Switching Power Supply 5V DC 0-5A 25W* permet d'avoir une tension de sortie à 5V $\pm 10\%$ pour arriver à 5.1V et de gérer la masse).

4. Utilisation de la DMA :

Pour éviter de surcharger le CPU avec la gestion de l'envoi des données vers le module PCM, la DMA peut directement communiquer avec elle pour gérer la transmission des données. L'initialiser et l'utiliser restent subtiles car il faut également comprendre la documentation. Nous avons dû faire en sorte que son fonctionnement soit similaire à celui de la STM32 avec un buffer circulaire via deux blocs de contrôles qui se pointent mutuellement pour que l'intégration soit plus simple. Il a fallu comprendre également que la Raspberry Pi possède plusieurs plages d'adresses différentes. Une pour le bus d'adressage du système, une accessible par le CPU ARM et un MMU permet de faire la translation entre ces deux plages. Donc lors de l'initialisation de la DMA, il faut utiliser les adresses de la plage accessible par le CPU et pour faire fonctionner la DMA, il faut utiliser la plage mémoire du bus système pour faire fonctionner la DMA pour laquelle accède au buffer et au PCM.

5. Intégration du code STM32 vers Raspberry Pi :

Dès que la Raspberry Pi 3B a fonctionné, Il a fallu faire un portage du code présent sur la STM32 pour le traitement du signal. Vu que la DMA fonctionne différemment et que le module PCM utilise des données de façon différente pour l'envoyer sur le bus I²S, il a fallu réarranger la structure de code. Faire en sorte de gérer la DMA de façon circulaire avec deux blocs de contrôles et de générer des interruptions à chaque moitié de tableau traitée par la DMA afin d'écrire les signaux numériques dans la partie du buffer de données disponible.

6. Optimisation nécessaire par le compilateur :

Lors des différents tests implémentés, un son saccadé a été remarqué lorsque 3 touches ou plus étaient appuyées. Lorsque les différents tests fonctionnels du programme ont permis de mettre de côté l'hypothèse d'un bug du programme, la question de la vitesse d'exécution a été mise en avant. Travaillant dans un environnement bare-metal, la charge des appels systèmes et des accès directement dans la RAM, sans aucune optimisation et sur un seul coeur, rendent le traitement Temps Réel compliqué. Afin de régler ce problème, une légère optimisation matérielle a été appliquée lors de la compilation gcc, avec le flag -O1. Le compilateur va essayer de produire un code plus rapide et plus compact sans prendre trop de temps de compilation et de taille mémoire.

7. Câblage instable :

Le dernier problème que nous avons rencontré est le câblage qui peut être instable. En effet, les câbles dupont peuvent bouger sur les broches, générant des bruits parasites lorsque le contact ne se fait pas correctement. Pour résoudre ce problème, nous avons élaboré une soudure entre la Raspberry Pi et le DAC externe permettant d'éviter cette instabilité.

Bilan

La réalisation de ce projet a permis de mobiliser de nombreuses compétences acquises au cours de notre Master, s'inscrivant pleinement dans la continuité de notre formation en systèmes embarqués.

Le travail mené s'est révélé particulièrement formateur, tant sur le plan organisationnel, à travers la gestion de projet, que sur le plan technique, en nous confrontant à des notions inconnues et en nous amenant à réinvestir les acquis de la formation.

Nous avons pu en apprendre davantage quant à de nouveaux protocoles de communications, qu'au fonctionnement de composants de microcontrôleurs, très prisés dans le domaine de l'embarqué.

Ainsi, ce projet transversal s'est révélé particulièrement intéressant, tant pour les apprentissages qu'il a permis que pour son caractère concret, dans un domaine qui a pu amuser l'ensemble de l'équipe.

Bibliographie

- [1] R. Stange, « Circle - C++ bare metal environment for Raspberry Pi ». [En ligne]. Disponible sur: <https://circle-rpi.readthedocs.io/en/50.0/index.html>
- [2] T. Instrument, « Texas Instruments SN74HC165/SN74HC165-Q1 8-Bit Shift Registers ». [En ligne]. Disponible sur: <https://eu.mouser.com/new/texas-instruments/ti-sn74hc165-parallel-load-shift-registers/>
- [3] C. M. \. T. Igoe, « Serial to Parallel Shifting-Out with a 74HC595 ». [En ligne]. Disponible sur: <https://docs.arduino.cc/tutorials/communication/guide-to-shift-out/>
- [4] Husamuldeen, « Working with STM32 and I2S Part 1 : Introduction ». [En ligne]. Disponible sur: <https://blog.embeddedexpert.io/?p=2553>
- [5] A. I42Heqce7f, « BCM2835 Audio Clock Management Guide ». [En ligne]. Disponible sur: <https://fr.scribd.com/doc/127599939/BCM2835-Audio-clocks>
- [6] T. Instrument, « 2VRMS DirectPath™, 112/106/100dB Audio Stereo DAC with 32-bit, 384kHz PCM Interface ». [En ligne]. Disponible sur: <https://www.ti.com/lit/ds/symlink/pcm5102.pdf?ts=1763572524050>
- [7] B. [Broadcom Corporation.], « BCM2837B0 Datasheet(PDF) 119 Page - Broadcom Corporation ». [En ligne]. Disponible sur: <https://www.alldatasheet.com/html-pdf/1572344/BOARDCOM/BCM2837B0/54257/119/BCM2837B0.html>
- [8] nh9, « Bare metal I2S input ». [En ligne]. Disponible sur: <https://forums.raspberrypi.com/viewtopic.php?f=72&t=275436>
- [9] B. Corporation, « BCM2837 ARM Peripherals ». [En ligne]. Disponible sur: <https://cs140e.sergio.bz/docs/BCM2837-ARM-Peripherals.pdf>
- [10] components101, « Raspberry Pi 3 ». [En ligne]. Disponible sur: <https://components101.com/microcontrollers/raspberry-pi-3-pinout-features-datasheet>
- [11] T. Instrument, « SNx4HC165 8-Bit Parallel-Load Shift Registers ».